

Bericht zur Belegaufgabe 16: Text-Clustering mit Spark

Anton Völker^{a,1} and Charnelle Campbell Djouela Teneku^{b,2}

^aFachbereich 4 Informatik, Kommunikation und Wirtschaft

^bFachbereich 4 Informatik, Kommunikation und Wirtschaft

Dr. Mykyta Kovalenko

Abstract—

Wir untersuchen Text-Clustering mit Apache Spark auf einem mehrsprachigen Korpus von rund 300 literarischen Texten in acht Sprachen. Die Pipeline umfasst Vorverarbeitung, TF-IDF und LDA. TF-IDF deckt fehlende Unicode-Normalisierung und zu grobe Tokenisierung auf und ergibt bei Einzeldokument-Läufen erwartungsgemäß $IDF = 0$. LDA liefert bei Zielgröße $K = 10$ weniger, vorwiegend sprachgetriebene Topics. Als nächste Schritte empfehlen wir Normalisierung und Hyperparameter-Suche für interpretierbare Cluster sowie eine semantische Topic-Benennung.

1. Introduction

Bevor die Transformer-Ära begann, prägten statistische und probabilistische Methoden die Textanalyse: mit TF-IDF für Suche und Ähnlichkeit sowie später LDA als generatives Themenmodell. Diese Verfahren skalieren über Dokumentengrenzen, benötigen kein Kontextfenster und eignen sich für verteilte Verarbeitung auf großen Korpora, etwa mit Spark. Daran knüpfen wir an und zeigen, wie eine Pipeline mit TF-IDF und LDA strukturierende Einblicke in einen mehrsprachigen Korpus liefert.

2. Term Frequency-Inverse Document Frequency (TF-IDF)

2.1. Grundlagen der TF-IDF

TF-IDF eingeführt durch [Salton and Buckley \(1988\)](#) ist eine numerische Statistik, die die Wichtigkeit eines Wortes in einem Dokument relativ zu einer Sammlung von Dokumenten widerspiegelt. Diese Sammlung von Dokumenten wird Korpus D genannt. Laut ([Manning et al., 2008](#), S. 127) ist TF-IDF eines der bekanntesten Gewichtungsschemata im Information Retrieval und Text Mining.

2.2. Mathematische Definition

Die TF-IDF eines Terms t in einem Dokument d aus einem Korpus D wird als Produkt zweier Komponenten berechnet:

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \times \text{idf}(t, D) \quad (1)$$

Term Frequency (TF)

Die Term Frequency beschreibt die Häufigkeit eines Terms in einem spezifischen Dokument. Eine häufig verwendete Definition ist:

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (2)$$

wobei $f_{t,d}$ die absolute Häufigkeit des Terms t im Dokument d ist und der Nenner die Gesamtanzahl aller Terme im Dokument darstellt. Alternativ kann auch die logarithmische Term Frequency verwendet werden:

$$\text{tf}(t, d) = \log(1 + f_{t,d}) \quad (3)$$

Inverse Document Frequency (IDF)

Die Inverse Document Frequency misst die Seltenheit eines Terms im gesamten Korpus:

$$\text{idf}(t, D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right) \quad (4)$$

wobei $|D|$ die Gesamtanzahl der Dokumente im Korpus ist und $|\{d \in D : t \in d\}|$ die Anzahl der Dokumente, die den Term t enthalten.

2.3. Interpretation und Anwendung

TF-IDF kombiniert die lokale Wichtigkeit eines Terms (TF) mit seiner globalen Seltenheit (IDF). Terme, die häufig in einem spezifischen Dokument, aber selten im gesamten Korpus vorkommen, erhalten hohe TF-IDF-Werte. Das ermöglicht charakteristische Terme für einzelne Dokumente zu identifizieren. Oft wird dies zur Feature Extraction in Machine Learning genutzt oder für Ähnlichkeitsberechnung zwischen Dokumenten. Laut [Blei et al. \(2003\)](#) reduziert TF-IDF die Beschreibungslänge nur gering und erfasst inter- und intradokumentare statistische Strukturen kaum. Deswegen wurde durch [Blei et al. \(2003\)](#) die Latent Dirichlet Allocation eingeführt die in kommende Absatz kurz behandelt wird.

3. Latent Dirichlet Allocation (LDA)

LDA, eingeführt durch [Blei et al. \(2003\)](#), ist ein Bayessches Netz zur Themenmodellierung in Texten. Es beschreibt, wie Dokumente als Mischungen latenter Themen generiert werden, wobei jedes Thema eine Wahrscheinlichkeitsverteilung über Wörter ist. „Latent“ bedeutet, dass die Themen nicht direkt beobachtbar sind, sondern aus der Wortverteilung im Text erschlossen werden.

3.1. Dirichlet-Verteilung

Die Dirichlet-Verteilung ist eine Verteilung über Mischungen. Für $K=3$ liegen diese Mischungen im Simplex (Dreieck). Der Parameter α steuert die Form: kleine α begünstigen dominante Einzelthemen (Ecken), $\alpha=1$ verteilt gleichmäßig, große α führen zu ausgewogenen Mischungen (Mitte).

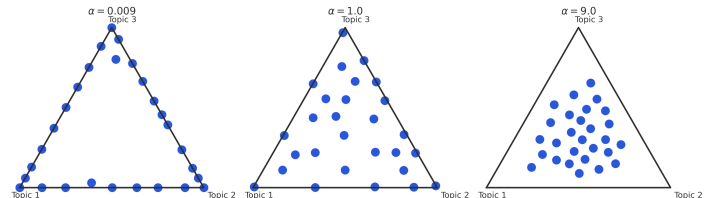


Figure 1. Intuition für die symmetrische Dirichlet-Verteilung $\text{Dir}(\alpha, \alpha, \alpha)$: Eckenhäufung bei kleinen α , Gleichverteilung bei $\alpha=1$, Zentrierung bei großen α .

3.2. Generatives Modell

$$P(W, Z, \theta, \phi \mid \alpha, \beta) = \left[\prod_{k=1}^K P(\phi_k \mid \beta) \right] \times \left[\prod_{d=1}^M P(\theta_d \mid \alpha) \prod_{n=1}^{N_d} P(z_{d,n} \mid \theta_d) P(w_{d,n} \mid \phi_{z_{d,n}}) \right] \quad (5)$$

Mit $P(\cdot)$ bezeichnen wir die bedingte Wahrscheinlichkeit/Dichte der beteiligten Größen. Die linke Seite in Gleichung (5) ist die *gemeinsame Wahrscheinlichkeit* von Wörtern W , Themenzuweisungen Z , Dokument-Mischungen θ und Themen-Wortverteilungen ϕ bei festen Hyperparametern α, β ; sie misst, wie gut das Modell den gesamten Korpus erzeugen kann.

Die rechte Seite beschreibt vier aufeinanderfolgende Schritte: (1) Für jedes Thema wird eine Wortverteilung ϕ_k festgelegt (welche

Wörter in diesem Thema typisch sind). (2) Für jedes Dokument wird eine Themenmischung θ_d festgelegt; α steuert, ob wenige Themen dominieren oder die Mischung ausgeglichen ist (vgl. Abb. 1). (3) Für jedes Wort im Dokument wird ein Thema $z_{d,n}$ entsprechend θ_d gewählt (Categorical). (4) Anschließend wird das Wort $w_{d,n}$ aus der zum gewählten Thema gehörenden Verteilung $\phi_{z_{d,n}}$ gewählt (Categorical). Der Ablauf ist in Algorithmus 1 zusammengefasst.

Algorithm 1 Generatives Verfahren von LDA (verteilungsbasiert)

```

1: for  $k = 1, \dots, K$  do                                ▷ Schritt (1): Thema → Wörter
2:    $\phi_k \sim \text{Dir}(\beta)$ 
3: end for
4: for  $d = 1, \dots, M$  do                                ▷ Schritt (2): Dokument → Themen
5:    $\theta_d \sim \text{Dir}(\alpha)$ 
6:   for  $n = 1, \dots, N_d$  do                            ▷ Schritte (3) & (4): je Token
7:      $z_{d,n} \sim \text{Cat}(\theta_d)$                             ▷ Categorical  $\equiv$  Multinomial mit  $n=1$ 
8:      $w_{d,n} \sim \text{Cat}(\phi_{z_{d,n}})$ 
9:   end for
10: end for
  
```

Ergebnisdarstellung: Die *Topics* sind die geschätzten ϕ_k (interpretierbar über ihre Top-Wörter), die *Dokument-Topics* sind die θ_d (Themenanteile je Dokument); optional zeigen $z_{d,n}$ die Themen auf Token-Ebene.

4. Daten

4.1. Korpusübersicht

Der für die Analysen verwendete Korpus besteht aus etwa 300 literarischen Texten in 8 verschiedenen Sprachen, die im Verzeichnis `/texts` organisiert sind. Die Verteilung der Dokumente nach Sprachen ist in Tabelle 1 dargestellt.

Sprache	Ungefähre Dokumentanzahl
Englisch	40
Französisch	40
Deutsch	35
Italienisch	30
Niederländisch	5
Spanisch	20
Russisch	60
Ukrainisch	30

Table 1. Verteilung der Dokumente nach Sprachen

4.2. Dokumenteigenschaften

Der Korpus umfasst verschiedene literarische Gattungen:

- Romane (z.B. "Pride and Prejudice" von Jane Austen)
- Kurzgeschichten (z.B. "The Adventures of Sherlock Holmes" von Arthur Conan Doyle)
- Theaterstücke (z.B. "Macbeth" von William Shakespeare)
- Gedichte (z.B. Werke von Paul Verlaine)
- Essays (z.B. "Essays" von Francis Bacon)

Die Texte stammen aus verschiedenen Epochen, von klassischen Werken bis zur Literatur des frühen 20. Jahrhunderts. Diese zeitliche Vielfalt bietet eine interessante Dimension für die Analyse der Themenentwicklung im Laufe der Zeit.

4.3. Preprocessing

Für unsere Themenmodellierung haben wir folgendes Preprocessing angewendet:

1. **Textextraktion:** Einlesen der Rohtexte mit Sparks [Zaharia et al. \(2016\)](#) `wholeTextFiles` unter Beibehaltung der Dokumentmetadaten (Dateipfad mit Sprachinformation)

2. **Tokenisierung:** Aufteilen des Textes in einzelne Wörter mittels Whitespace-Trennung
3. **Kleinschreibung:** Umwandlung aller Texte in Kleinbuchstaben mit `text.lower()`
4. **Stoppwort-Entfernung:** Entfernung häufiger Stoppwörter spezifisch für jede Sprache mittels der `nltk` Library von ([Loper and Bird, 2002](#))

5. Implementierung

Unsere Implementierung folgt dem funktionalen Programmierparadigma ([Backus, 1978](#)), das besonders gut zu Spark's ([Zaharia et al., 2016](#)) verteilter Datenverarbeitung passt. Die Kernfunktionalität ist in vier Python-Modulen organisiert:

- `util.py`: Hilfsfunktionen für Datenladung und -vorverarbeitung
- `tf_idf.py`: Implementierung des TF-IDF-Algorithmus
- `lda.py`: Funktionen für die LDA-Themenmodellierung
- `main.py`: Orchestrierung der Pipeline

6. Experimente

Wir haben verschiedene Experimente durchgeführt, um die Effektivität unserer TF-IDF- und LDA-Implementierungen auf dem mehrsprachigen Korpus zu evaluieren. Dabei haben wir sowohl mit Teilmengen als auch mit dem vollständigen Datensatz gearbeitet. Alle Experimente wurden auf einem MacOS-System mit Apache Spark in lokaler Ausführung durchgeführt. Die Verarbeitung erfolgte mit folgenden Dependencies:

- **Hardware:** MacOS-System mit x86_64-Architektur
- **Spark-Konfiguration:** Lokaler Modus mit dynamischer Ressourcenzuweisung
- **Python-Version:** 3.13
- **Bibliotheken:** PySpark 3.5.0

TF-IDF

Wir haben zwei Berechnungen durchgeführt: (i) auf einer Teilmenge, genauer eines einzelnen Dokuments (Baas Gansendonck - Hendrik Conscience.txt) zu Entwicklungszwecken und (ii) auf dem vollständigen Korpus. Zur Auswertung betrachteten wir die zehn höchsten TF-IDF-Scores.

LDA

Analog gingen wir bei LDA vor. Für ein einzelnes Dokument führten wir drei Iterationen mit zehn Topics durch. Für den gesamten Korpus wählten wir die Anzahl der Topics erneut mit zehn und setzten die Zahl der Iterationen auf zehn.

7. Ergebnisse

TF-IDF

Die Ergebnisse sind für das Experiment mit nur einen Dokument für alle Top-10-Werte 0.0, weil wir IDF wie folgt berechnen:

```

1 n_docs_subset = docs_subset.count()
2 idf_subset = df_subset.mapValues(lambda df_val: math.log
   ↪ (n_docs_subset / df_val))
3 idf_dict_subset = dict(idf_subset.collect())
4 idf_broadcast_subset = sc.broadcast(idf_dict_subset)
5 tf_idf_subset = calc_tf_idf(tf=tf_subset, idf_broadcast=
   ↪ idf_broadcast_subset)
  
```

Wenn das Subset nur ein Dokument enthält (`n_docs_subset=1`) bzw. alle Terme in allen Subset-Dokumenten vorkommen (`df_val=n_docs_subset`), ergibt sich $IDF = \log(1) = 0$ und damit

$TF\text{-}IDF = 0$ trotz positiver TF. Dementsprechend ist das Experiment nicht aussagekräftig, bestätigte aber zunächst eine fehlerfrei durchlaufende Implementierung; durch das z. B. ausschließliche Auftauchen niederländischer Wörter wurde klar, dass das Laden der Daten funktionierte, und wie gesagt war das Hauptziel auch, eine Unterstützung in der Entwicklung zu haben. Für das zweite Experiment ergaben sich die folgenden Top-10-Wörter:

Wort	TF-IDF-Score
–	17048.22
–	11322.83
á	10651.45
–	10326.36
!...	9814.15
á	7256.03
;	7109.32
d'artagnan	6265.75
dit	5853.69
eulenspiegel	5570.21

Table 2. TF-IDF-Topwörter und ihre Scores

Die sehr hohen TF-IDF-Werte für Satzzeichen wie „–“, „!...“ oder „;“ zeigen, dass in der Vorverarbeitung Fehler aufgetreten sind: Es fehlt die Unicode-Normalisierung, und die Tokenisierung ist zu grob und ohne Zeichensetzungsfilter, wodurch Satzzeichen fälschlich als Wörter gezählt und ähnliche Zeichen (z. B. „–“ und „–“) nicht als identisch behandelt wurden.

LDA

Das erste Experiment lieferte weniger Topics als die Zielvorgabe ($K = 10$). Die Top-Wörter sind durchgängig niederländisch; die Topics bilden folgerichtig ausschließlich niederländische Vokabeln ab. Das ist erwartbar, da der zugrunde liegende Korpus hier faktisch nur ein niederländisches Dokument enthält.

topic	termsWordsStr
7	kerkhorf, beven?, zuken, gloeien, droomde, schoenen, twijgen, gal, nengens, klaar
5	langs, hulde, scherp, laffe, goud, blik, stuursch, 1830, slacht, goedkeuring
6	lisa, schuimer, plichten, toegelopen, groeien, aanboud, lampaards, officieren, lachen, god.
2	opgestaan, streek, hongersnood, kruiden, boomen, persoon, schreeuwden, gereedgemaakt, uitnoodiging
NaI	god\\\"
zaten	dreeven

Figure 2. Ergebnisse Experiment 1

Auch das zweite Experiment ergab weniger als $K = 10$ Topics. Die Top-Wörter decken alle vorhandenen Sprachen ab; die Modellierung segmentiert primär nach Sprache, d. h. die Topics fallen als sprachbasierte Cluster aus.

topic	termsWordsStr
0	belcredi, si, landolfo, baldovino, matilde, cosi, fabio, maurizio, ciò, nolli
1	thir, ilse, sagte, onnen, thy, heav'n, th', xich, dr.
2	t, zee, maor, ie, as, veur, daor, gwynplaine, zoo, jaaije
3	d, vd., usted, tan, pepe, tirso, ser, después, doña, condesa
4	egli, disse, senza, cosi, ella, quando, poi, quel, cosa
5	would, said, could, one, mr., upon, little, much, must, us
6	i., plus, qu'il, cette, comme, tout, si, i
7	sagte, ilse, schon, ganz, mehr, xich, immer, wohl, ja, hatte,
8	i, wo, a, ra, xk, ilse, a, onnen, he, na

Figure 3. Ergebnisse Experiment 2

8. Fazit

TF-IDF hat aufgezeigt, dass Fehler in der Entwicklung aufgetreten sind, was an sich eine Stärke der Methode aufzeigt, da die Computezeit relativ gering war aber aussagekräftig Fehler offenlegte. Eine semantische Bedeutung des Textes konnte daraus jedoch nicht extrahiert werden. LDA hat grob gezeigt, dass eine Modellierung von

Topics möglich war, wobei einige Indikatoren (z. B. abweichende Topic-Anzahl) stutzig machten und eine weitere Untersuchung anregen würden. Außerdem wäre eine semantische Benennung der Topics sowie eine feingranulare Untersuchung der einzelnen Cluster interessant: Welche semantische Bedeutung haben die Cluster neben der offensichtlichen Unterscheidung nach Sprache? Ein weiteres, systematischeres Experimentieren mit den Hyperparametern wäre ebenfalls interessant gewesen. Bspw. wäre es interessant die Anzahl der Topics zu reduzieren sodass $K < \text{Anzahl der Sprachen}$ ist um eine Gruppierung über Sprachen hinweg entlang der semantischen Bedeutung zu provozieren. Die relevanten Bereiche wurden mit den gegebenen Möglichkeiten bearbeitet. Umfang und Qualität der Experimente hätten sich mit mehr Ressourcen voraussichtlich weiter steigern lassen.

Source Code

<https://github.com/antonallote/ProKo>

References

- John Backus. Can programming be liberated from the von neumann style? a functional style and its algebra of programs. *Communications of the ACM*, 21(8):613–641, 1978.
- David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022, 2003.
- Edward Loper and Steven Bird. NLTK: The natural language toolkit. In *Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics*, pages 63–70, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics. doi: 10.3115/1118108.1118117. URL <https://aclanthology.org/W02-0109/>.
- Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008.
- Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523, 1988.
- Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. Apache spark: a unified engine for big data processing. *Commun. ACM*, 59(11):56–65, October 2016. ISSN 0001-0782. doi: 10.1145/2934664. URL <https://doi.org/10.1145/2934664>.